

SIMPLE USER GUIDE / v3.13

XEMU RAW Cheat Codes

What each code type does, when to use it, and how to read a working example.

All addresses in this guide are teaching examples. They are not tested addresses for any game. Replace them with addresses and values verified for the exact game revision you are using. Back up important saves before experimenting.

Start with the result you need

I want to...	Use	Page
Keep a value such as health or money at a chosen number	0 / 1 / 2	3
Add or subtract, fill a series, or copy bytes	3 / 4 / 5	4-5
Follow an address that moves	6	6
Change individual on/off bits	7	7
Use a full address, physical memory, or a base pointer	9	8
Run a code only when a condition matches, or toggle it	D / E	9-10
Write an exact sequence of bytes	A	11
Replace game instructions with a custom routine	F0 / F1 / F2	12-15
Apply a startup/reset patch instead of a repeating cheat	PREENTRY	16

Three ideas to keep in mind

An address is where data lives. **A value** is what you want there. **A code type** tells the engine what action to perform. A pointer is simply an address stored in memory.

Most normal cheats run repeatedly while enabled. A code that writes a fixed number keeps putting that number back; an arithmetic code keeps adding or subtracting. These are different uses.

New users: read pages 2-3 first. F0/F1/F2 are advanced tools; they require an understanding of the original game instructions at the hook address.

GETTING STARTED

Create and load a cheat file

Use the file XEMU creates for your game

Open **Debug > Cheat Engine**, then **File > Edit/Add Cheats**. This opens the matched file, or creates a file with the current game header. Keep that generated header. Save your edits, then use **File > Reload** or **Reload (Cheat File)**.

```
^1 = Hash: <keep the hash generated by XEMU>
^2 = GameID: <keep the generated GameID>
^3 = NAME: <your game name>

!Player Codes:
+Example Health{Set the verified health value to 100}
%Credits: Your Name
$000001000 000000064
!!
```

The angle-bracket text above explains the header; it is **not text to paste literally**. The example health address is also a placeholder.

Line	Meaning
^1 / ^2 / ^3	Hash, GameID, and display name. Let XEMU create these.
!Player Codes: / !!	Open and close a group. Groups may be nested.
+Example Health{...}	Create a selectable code, with an optional description.
%Credits: Your Name	Credit the code author.
\$XXXXXXXXX YYYYYYYY	A RAW code line. The leading \$ is optional.
; Comment on its own line	Use a separate semicolon comment line. For a comment after a RAW pair, use // or # instead.

On the **Cheats** tab, switch **Disabled** to **Enabled**, then tick the desired code. **Options > Engine Enabled** must also be on. Existing persistent selections may affect the starting state.

Files live in the **Cheats** directory beside the executable, with a relative Cheats-path fallback only if executable discovery fails. **File > Open** opens the actual directory. Headers, not just filenames, identify the game. New files use normalized uppercase hashes; legacy raw-header matches remain supported.

BASIC CODES

0 / 1 / 2: write a value

These types write 1, 2, or 4 bytes. Choose the size used by the game, not simply the biggest size available. Writing too many bytes can overwrite nearby data.

Type	Size	Format
0	8-bit / 1 byte	0AAAAAAA 000000VV
1	16-bit / 2 bytes	1AAAAAAA 0000VVVV
2	32-bit / 4 bytes	2AAAAAAA VVVVVVVV

Three independent examples

```
$00001000 00000064
```

Write **100 decimal** (hex **64**) to virtual address **00001000**.

```
$10001010 000003E7
```

Write **999 decimal** (hex **03E7**) to virtual address **00001010**.

```
$20001020 000F423F
```

Write **999,999 decimal** (hex **000F423F**) to virtual address **00001020**.

Reading the numbers

RAW numbers are **hexadecimal**: digits 0-9 and A-F. For example, hex 10 is decimal 16, not decimal 10. One byte fits 00-FF; two bytes fit 0000-FFFF; four bytes fit 00000000-FFFFFFFF.

The leading digit is the **type**, not part of the address. Ordinary address fields have seven hex digits. Use **Type 9** for a full 32-bit base such as 81200000. The default is **guest virtual memory**; XEMU does not silently add 80000000.

Integer writes are little-endian: writing the 32-bit value 11223344 stores bytes 44 33 22 11. Type A instead writes bytes in the order shown. Disabling a basic write stops future writes; it does not automatically restore the previous value.

ARITHMETIC

3: add or subtract

Type 3 reads the current value, changes it, and writes it back. The address is in the **second word**, unlike a basic write.

Action	Format
Add to an 8-bit value	300000VV 0AAAAAAA
Subtract from an 8-bit value	301000VV 0AAAAAAA
Add to a 16-bit value	3020VVVV 0AAAAAAA
Subtract from a 16-bit value	3030VVVV 0AAAAAAA
Add to a 32-bit value	30400000 0AAAAAAA Next line: VVVVVVVV 00000000
Subtract from a 32-bit value	30500000 0AAAAAAA Next line: VVVVVVVV 00000000

Add 1 to a byte

```
$30000001 00001000
```

If address 00001000 holds decimal 10, this execution changes it to 11.

Subtract 10 from a 16-bit value

```
$3030000A 00001010
```

Hex 000A is decimal 10. A starting value of 100 becomes 90.

Add 100 to a 32-bit value

```
$30400000 00001020
$00000064 00000000
```

The second line supplies the amount: hex 64 is decimal 100. A starting value of 1,000 becomes 1,100.

This repeats while enabled. Do not use Type 3 when you only want to set a fixed value once. Arithmetic wraps at its size limit: adding 1 to an 8-bit FF produces 00.

MULTI-LINE CODES

4 / 5: series and copies

Type 4: write a sequence of 32-bit values

```
4AAAAAAA NNNNSSSS
VVVVVVVV IIIIIIII
```

A is the starting address. **N** is the number of writes. **S** is the address step in groups of four bytes. **V** is the first value; **I** is the amount added to the value after each write.

```
$40001000 00030001
$00000064 0000000A
```

Address	Value (hex)	Value (decimal)
00001000	00000064	100
00001004	0000006E	110
00001008	00000078	120

Here N=0003 means three writes, S=0001 means move four bytes, and I=0000000A means add decimal 10. Use I=00000000 to repeat the same value.

Type 5: copy bytes from one address to another

```
5AAAAAAA NNNNNNNN
DDDDDDDD 00000000
```

```
$50001000 00000008
$00002000 00000000
```

Copy **8 bytes** from 00001000 to 00002000. The second line contains the destination, not another cheat command. Source and destination both use the active Type-9 base/address space when one is active.

Keep both lines together. Type 5 has a **16 MiB per-command safety limit**. It reads the source before writing the destination, so this is not a byte-by-byte forward copy. Large repeated copies can still be expensive.

MOVING ADDRESSES

6: follow a pointer

A pointer is an address stored at another address. Type 6 follows that pointer and then writes a value. This is useful when a game object moves between levels or sessions.

```
6AAAAAAA VVVVVVVV
000TNNNN 00000000
00000000 00000000 // extra offsets only when needed
```

A holds the first pointer. **V** is the value to write. **T** is 0 for 8-bit, 1 for 16-bit, or 2 for 32-bit. **N** is the number of offsets. The first offset is on line 2; remaining offsets are packed two per line.

One offset: pointer + 20

```
$60001000 000003E7
$00020001 00000020
```

Assume 00001000 contains pointer **80002000**. Add hex 20 and write 999 at **80002020**. The full 32-bit pointer is kept.

Two offsets: follow another pointer, then add 4

```
$60001000 000003E7
$00020002 00000020
$00000004 00000000
```

Step	Assumed memory / result
Read the root	[00001000] = 80002000
Add 20, read again	[80002020] = 80003000
Add the final 4, write	Write 999 to 80003004

N counts offsets, not source lines. Each offset except the last leads to another 32-bit pointer read. The last offset chooses the write address. Type 9 controls the address space for every pointer read and the final write.

Use N=0001 or more; 0000 is accepted as a legacy one-offset form. The limit is 256 offsets. A null pointer or failed memory read prevents that pointer write. Always verify the pointer chain for the game revision.

FLAGS

7: change individual bits

Some values pack several on/off flags into one byte or 16-bit number. Type 7 can alter selected bits without replacing the whole value.

```
7AAAAAAA 00T0VVVV
```

T	Operation	In plain language
0 / 1	8-bit / 16-bit OR	Set the bits that are 1 in your mask.
2 / 3	8-bit / 16-bit AND	Keep only the bits that are 1 in your mask.
4 / 5	8-bit / 16-bit XOR	Flip the bits that are 1 in your mask.

Set bit 2 without changing the other bits

```
$70001000 00000004
```

Mask 04 selects bit 2, counting bits from 0. If the byte starts at 01, it becomes 05.

Clear bit 2 without changing the other bits

```
$70001000 002000FB
```

Mask FB has every byte bit set except bit 2. If the byte starts at 0F, it becomes 0B.

Flip bit 2

```
$70001000 00400004
```

If the byte starts at 01, it becomes 05. Running the same XOR again changes 05 back to 01.

XOR is not an automatic one-press toggle. A live XOR code flips the selected bits every time it executes. Use a carefully designed condition or one-shot patch when repeated flipping is not intended.

For a 16-bit clear-bit-2 operation, select T=3 and use mask FFFB. Do not use an 8-bit mask when you intend to keep the other bits of a 16-bit value. These are integer bit operations, not floating-point math.

BASES & ADDRESS SPACES

9: choose an address context

```
9MCCCCC BBBB BBBB
```

M chooses a mode. **C** counts following RAW entries in this scope; 000000 means to the end of the current cheat block. **B** is a full 32-bit base or pointer address.

M	Meaning
0	Virtual: use B as the base.
1	Physical: use B as the base.
2	Virtual: read a 32-bit pointer at B; use it as the base.
3	Physical: read a 32-bit pointer at B; use it as the base.

Full virtual base

```
$90000002 81200000
$20000020 000003E7
$10000024 00000064
```

Write 999 at **81200020** and 100 at **81200024**. The two following RAW lines use base 81200000.

Physical base

```
$91000001 00001000
$20000020 000003E7
```

Write 999 at **physical 00001020**, not at that virtual address.

Pointer base

```
$92000001 00001000
$20000020 000003E7
```

Read a pointer from virtual 00001000. If it is 80002000, write 999 at 80002020. Use **93000001** instead of 92000001 to read and write in physical memory.

Nested scopes add B to the current base. Count entries **after the header**, including continuation data; a captured Type-F body counts as one. Use C=0 only when all remaining codes should share this context.

D/E use the active context. F0/F1/F2 need virtual mode.

CONDITIONS / PART 1

D / E: read the condition fields

```
Daaaaaaa NNTsvvvv // D: run while the test matches
Eaaaaaaa NNTsvvvv // E: toggle on a new matching edge
```

a is the address to check. **NN** is the amount of following code guarded. **T** is the comparison. **s** selects size and address space. **vvvv** is the value to compare against.

T	Test	Meaning
0	memory == value	Equal
1	memory != value	Not equal
2	memory < value	Less than
3	memory > value	Greater than
4	(memory & value) == 0	None of the mask bits is set
5	(memory & value) != 0	At least one mask bit is set
6	(memory value) == 0	Both values are zero
7	(memory value) != 0	At least one value is nonzero

s	Read size and space
0	16-bit physical
1	16-bit virtual
2	8-bit physical
3	8-bit virtual

D example: write only while a byte equals 1

```
$D0001000 01030001
$20002000 000003E7
```

Read virtual byte 00001000. If it equals 1, write 999 to virtual 00002000. Otherwise, skip that write. Here **NN=01**, **T=0**, **s=3**, **vvvv=0001**.

Comparisons are unsigned 8-bit or 16-bit, not signed or floating-point comparisons. Outside Type 9, **s** selects only the comparison read; it does not change the space used by the following write. Inside Type 9, the active base and space override the D/E address space.

CONDITIONS / PART 2

E toggles and safe skip counts

E example: toggle a guarded write on and off

```
$E0001000 01030001
$20002000 000003E7
```

Use a verified 0/1 flag, such as a game-specific input flag. A change from **not matching** to **matching** toggles an internal switch. The write runs while that switch is on, even after the flag is released.

Flag at 00001000	Switch	Guarded write
0: first observation	Off	Skipped
1: new match	On	Runs
1: still held	On	Runs; no extra toggle
0: released	On	Still runs
1: next new match	Off	Skipped

A matching condition on its first evaluation counts as a new match. A memory-read error does not trigger a new toggle and prevents that execution of the guarded code.

Count the right things

Options > Code-Type-Aware D/E (experimental) is off by default. With it off, NN counts literal parsed RAW entries, including continuation data. With it on, NN counts logical commands, including their owned data/scope.

Default mode: guard a two-line Type 4 with NN=02

```
$D0001000 02030001
$40002000 00030001
$00000064 0000000A
```

The Type-4 header and its data line are both skipped when the condition is false. In code-aware mode, a count of 01 can skip the complete Type-4 command instead.

Do not count group names, comments, or blank text lines. F0/F1/F2 are special: the parser captures the complete body as **one RAW entry**. In code-aware mode, skipping Type 9 can skip its whole scoped group, not just the header. Keep the file and option setting consistent.

EXACT BYTE ORDER

A: write literal bytes

Type A is for a sequence of bytes, such as a verified machine-code patch or a small data block. It is XEMU-specific; do not assume another device or emulator uses Type A the same way.

```
AXXXXXXX ZZZZZZZZ
DDDDDDDD DDDDDDDD
... more data pairs only if needed ...
```

X is the destination address or current-context offset. **Z** is the total number of bytes. Every continuation pair supplies eight bytes, read from left to right. Add enough data lines to cover Z.

Write exactly six bytes

```
$A0001000 00000006
$DEADBEEF 11220000
```

```
Bytes written: DE AD BE EF 11 22
Unused tail:   00 00 (ignored)
```

The example writes six bytes starting at 00001000. The last two bytes of the data pair are beyond the requested length and are not written.

How many data lines?

Byte count (hex)	Bytes (decimal)	Required data lines
00000001 to 00000008	1 to 8	1
00000009 to 00000010	9 to 16	2
00000011 to 00000018	17 to 24	3

Byte order matters. Type A data DEADBEEF 11220000 starts DE AD BE EF. A basic 32-bit write of DEADBEEF instead stores EF BE AD DE. These formats are not interchangeable.

A zero byte count is an error. Missing data lines are an error. Always keep continuation data with its header, especially under D/E conditions. The address space comes from Type 9 when a context is active.

Types not supported here

8, B, and C are not implemented as Xbox memory operations. Do not copy setup/master-code meanings from another cheat engine and expect them to work. The engine reports unsupported RAW types.

ADVANCED / AUTOMATIC CODE CAVES

F0: write a custom x86 routine

A **hook** redirects the game from an instruction address to a small custom routine. A **code cave** is the extra space used for that routine. XEMU allocates the cave; you do not need to reserve an ordinary game-RAM address for it.

```
$F0000000 AAAAAAAA  
$<Intel-syntax instructions and labels>  
$DEADCODE
```

A is the full virtual hook address, or an offset inside a virtual Type-9 context. The low 24 bits of the F0 command must stay zero. **DEADCODE** ends the executable section; XEMU generates a jump back.

A small example with an explicit assumption

Suppose the verified instruction at 00010000 is exactly a five-byte **mov eax, immediate**, and replacing its result with 999 is the intended change:

```
$F0000000 00010000  
$mov eax,0x000003E7  
$DEADCODE
```

The custom routine puts 999 into EAX. The generated return goes to the first instruction after the overwritten hook span. This deliberately changes EAX; saving and restoring EAX here would undo the result.

Inspect the original instructions first. XEMU covers whole instructions until it has room for its jump. Your routine must reproduce any displaced instructions whose effects are still needed. DEADCODE does **not** automatically replay those instructions.

F0 writing rules

Use Intel syntax, explicit operand sizes where needed, and **0x** on absolute numeric memory addresses, for example **mov cl,byte ptr [0x0046D784]**. Bare numeric literals are hex-first; use 0x0000000A for decimal 10.

F0 supports labels and branch targets. It also has DD data, PRESERVE/RESTORE, and private T0-T7 scratch registers. Those are shown next. F hooks need virtual addressing, not a physical Type-9 context.

The cave mapping is separate from normal Xbox machine RAM. Hook installation and safe teardown are managed by XEMU. A hook is not a repeating ordinary write: its routine executes when the game reaches that instruction.

ADVANCED / F0 HELPERS

F0: preserve registers and use data

Keep a temporary register change from leaking into the game

```
$F00000000 00010000
$PRESERVE EAX
$mov eax,dword ptr [Value]
$mov dword ptr [0x00002000],eax
$RESTORE
$DEADCODE
$Value:
$dd 000003E7
```

This payload reads the attached value 999, writes it to virtual 00002000, and restores the original EAX. **Value** is a local label; DD emits a 32-bit little-endian value. Label/DD data may follow DEADCODE and stays attached to that cave.

This demonstrates payload syntax, not a verified complete game hook. At your chosen hook address, also preserve or recreate any original instruction effects the game still needs.

Directive	Meaning
PRESERVE EAX, ECX, EFLAGS	Save only the named registers/flags.
PRESERVEALL	Save EAX, EBX, ECX, EDX, ESI, EDI, EBP and EFLAGS.
RESTORE	Restore everything still saved in the current preserve frame.
RESTORE EAX	Restore only the named saved register.

ESP and EIP are not supported PRESERVE targets. Balance save/restore paths, including branches. Do not restore a register when its new value is the intended output of your hook.

T0-T7: private scratch values

```
$F00000000 00010000
$add T0,0x00000001
$DEADCODE
```

This example increments a private counter each time the hook runs. T0-T7 are not real Xbox CPU registers. Each T-using F0 owns a persistent private bank until it is disabled/reset. Use them for temporary state without dedicating EAX or another game register.

T-based flag-writing operations capture results in **TFLAGS** while restoring the game flags; following supported conditional branches use that private result. This is advanced behavior, not a substitute for checking the complete hook logic.

ADVANCED / FIXED PAIRS

F1: supply the raw x86 bytes

F1 is the legacy raw-byte form of an external hook. Use it when you already have correctly encoded x86 bytes. F0 is usually easier to read and maintain.

```
$F1000000 AAAAAAAA
$XXXXXXXX YYYYYYYY
$DEADCODE 000000NN
```

Each payload line contains **two eight-digit words**, representing eight literal bytes. NN is 01-08: the number of valid bytes in the **last** payload line. It is not the total cave size.

The same mov-eax-999 example

```
$F1000000 00010000
$B8E70300 00000000
$DEADCODE 00000005
```

```
Use:      B8 E7 03 00 00  // mov eax, 000003E7
Padding: 00 00 00        // not written as payload
```

Only five bytes from the last line are used. XEMU then appends the generated return jump immediately after those five bytes.

Situation	What to write
All 8 bytes of the last pair are meaningful	DEADCODE 00000008
Only 1-7 bytes of the last pair are meaningful	Use that byte count; every unused tail byte must be 00.
Payload needs several pairs	Every earlier pair contributes all 8 bytes; NN affects only the final pair.

F1 writes bytes **exactly as typed**. A raw CALL/JMP displacement is not automatically relocated to a label. Use F0 or F2 when you need labels and relocated branches.

F1 shares the F0 hook rules: a virtual address, whole-instruction overwrite coverage, and a generated return after the original hook span. You must reproduce any original instruction effects that remain necessary. The same game-specific verification is required.

A nonzero unused tail, invalid NN, missing DEADCODE, malformed pair, or oversized payload produces an error. The F1 parser limits executable payload to 64 KiB.

ADVANCED / RELOCATABLE RAW FORM

F2: raw bytes with labels

F2 keeps raw bytes readable while allowing XEMU to resolve supported label references. Unlike F1, it does not require every line to be an eight-byte pair.

Literal bytes

```
$F2000000 00010000
$B8E7030000
$DEADCODE
```

This has the same mov-eax-999 payload as the F0/F1 examples. There is no tail count after F2 DEADCODE. Spaces between byte groups are allowed.

A label, a branch, and attached data

```
$F2000000 00010000
$B8 Value
$8B00
$E9 Done
$Done:
$90
$DEADCODE
$Value:
$DD 000003E7
```

B8 Value loads the address of Value into EAX. **8B00** reads the 32-bit value through EAX. **E9 Done** jumps to the final NOP (90), then returns. DD stores 999 after the generated return jump; its address is resolved by XEMU.

Supported label forms	Meaning
E8 / E9	Near relative CALL / JMP
EB / 70-7F / E0-E3	Short jump / condition / loop forms
0F80-0F8F	Near conditional jumps
B8-BF / 68 / A0-A3	Supported absolute-32 immediate/address forms

Raw bytes stay literal. To force a hex-looking word to be a label, use @, such as **E9 @FACE** with a label named FACE. A short branch still has a limited reach; use an appropriate near form when needed.

F2 is not an Intel-instruction text parser. Write mov/cmp-style instructions in F0, and raw opcode bytes in F2. The same original-instruction, virtual-address, and safe-hook requirements apply.

USING CODES SAFELY

PREENTRY, disabling, and saved choices

PREENTRY belongs on the Patch tab

Use the special PREENTRY prefix for a code intended to apply once during startup/reset handling, rather than repeatedly as a live cheat. It is a block marker, not a new hexadecimal code type.

```
+:PREENTRY:Example startup value{Requires reset}
$20001020 000003E7
```

Select it on the **Patch** tab, then reset the game. Watch the status: **RESET REQUIRED** means the selection change has not taken full effect yet. Deselecting an applied patch also requires a reset to return to an unpatched startup state.

Keep live cheats and startup patches separate. Selecting a PREENTRY entry does not run it immediately like an ordinary live code. Do not use PREENTRY as a substitute for verifying when the game actually initializes that value.

What happens when you disable a code?

Code family	Effect of disabling
Ordinary memory write / arithmetic / copy / bitwise	Stops future executions. Already-written values are not automatically rolled back.
Live F0 / F1 / F2 hook	XEMU manages restoration of original hook bytes and safe cave retirement. Data changed by the routine is not automatically undone.
Applied PREENTRY patch	Change the Patch selection and reset the game. The Patch status tells you when reset is required.

Persistent Cheats

Options > Persistent Cheats remembers selections for the game/revision. It is off by default unless previously enabled. It does not make an incompatible code safe or turn a memory patch into a save-state rollback.

When using separate ON and OFF blocks, uncheck ON before running OFF. The engine does not automatically make ordinary cheat checkboxes mutually exclusive. Only use an OFF value you have verified as the correct original value.

A code can change data that the game later saves. Stopping the code cannot undo a save that has already been written. Keep backups of important game saves.

WHEN A CODE DOES NOT DO WHAT YOU EXPECT

Troubleshooting

Symptom	Check
The cheat file does not load	Keep the generated Hash and GameID header. Reload the current file. Confirm the game revision matches; a similar filename alone is not enough.
The code appears, but nothing happens	Enable Engine Enabled and the Cheats tab master switch. Tick the code. For PREENTRY, use Patch and reset.
The value changes once, then returns	The game may overwrite it, use another address, or keep a second authoritative value. Confirm the actual runtime address and correct size.
Arithmetic or XOR changes too quickly	These execute repeatedly while live. Use a basic fixed-value write, a properly scoped condition, or a suitable one-shot patch.
A pointer code fails after a level change	Recheck every pointer read and offset. Verify virtual versus physical memory and the exact game revision.
D/E skips the wrong code	Check NN, continuation entries, and the Code-Type-Aware option. A captured F block is one RAW entry, not one entry per text line.
A raw byte patch is reversed	Do not confuse little-endian integer writes with literal Type-A/F1/F2 byte order.
An F0 absolute memory operand fails	Use an explicit 0x address and operand size when needed, e.g. <code>mov cl,byte ptr [0x0046D784]</code> .
An F hook crashes the game	Check the original full instruction span, registers/flags, branch targets, and displaced instruction effects. Disable it and return to a known-good state.
Runtime or parse message appears	Read the reported source line and error. Check data counts, supported type/subtype, byte padding, and DEADCODE.

Change one thing at a time. Start with one small code, watch the intended address, and confirm that unrelated values remain unchanged. Do not enable a large group of unverified examples together.

For a useful problem report, include the game/revision identity, complete code block, enabled options, error text, and whether it happens at startup or after a level change. Do not share private save data unnecessarily.

REFERENCE

Quick reference and source notes

Type	Purpose	Key detail
0 / 1 / 2	8 / 16 / 32-bit write	Default virtual address; fixed value.
3	Add / subtract	Address is in word 2; repeats while live.
4	Serial 32-bit writes	Address step is S multiplied by 4.
5	Copy bytes	Header + destination line; 16 MiB limit.
6	Pointer write	Offsets, not lines; maximum 256.
7	Bitwise OR / AND / XOR	Byte or 16-bit; XOR repeats.
9	Address/base context	Mode 0-3; C=0 runs to block end.
A	Literal byte write	Read bytes left to right; enough pairs for Z.
D / E	Condition / edge toggle	NN count depends on skip mode.
F0	Assembly hook	Intel syntax; DEADCODE without operand.
F1	Raw-pair hook	DEADCODE NN = 01-08 final valid bytes.
F2	Raw/label hook	Relocatable labels; no DEADCODE operand.
8 / B / C	Not supported	No Xbox memory operation is implemented.

This guide follows the packaged implementation

Source baseline: user-confirmed v3.12, with the behavior-preserving v3.13 pointer-offset cleanup. In **ui/xui/debug-tools/**: **cheat-engine-source.cc** defines file parsing; **cheat-engine-execute.cc** defines RAW operations, scopes and D/E; **cheat-engine-fhooks.cc** defines F lifecycle and raw parsing; **x86-cheat-assembler*.cc** define F0/F2; **cheat-engine-ui.cc** and **cheat-engine-persistence.cc** define the controls and saved selections.

The packaged tests include the numerical examples, all D operators and size/space selectors, E edge transitions, scope/skip cases, and an exact-v3.12 Type-6 comparison. The tests execute the actual parser/executor functions against an isolated memory backend; they do not validate any game address. F1 bytes and F2 label/data examples are checked locally. F0 examples were reviewed against the implementation; full Keystone assembly was not tested in this environment.

Examples are teaching material, not a game-code database. A code is usable only after its address, size, value, and timing are verified for the intended game build.